Projukti Sheba

ENGINEERING HIRING ASSESSMENT

Position: Laravel / Full-Stack Engineer
Estimated Effort: 24–48 Hours
Practical Coding Challenge

01. Challenge Objective

> Build a standalone, clean Laravel mini-application titled **"Task & Time Tracker"**. Demonstrate your software craftsmanship across database modeling, Eloquent relationships, state resilience, front-end interactivity, and clean architecture.

02. Core Functional Requirements

A Task Checklist / Sub-Tasks

- Users can create tasks and access a dedicated **Task Detail View**.
- Inside a task, users can add and remove multiple checklist items (sub-tasks).
- Checklist items can be marked complete/incomplete. Seamless updates (without a full page reload) are strongly preferred.
- A **dynamic progress indicator** must automatically reflect the completion rate of checklist items.

B Live Stopwatch & Worklogs

- An interactive real-time stopwatch widget on the task view (**Start**, **Pause**, **Stop & Save**).
- **Resilience:** The stopwatch must not lose elapsed time if the user refreshes or reloads the browser window.
- Stopping the timer prompts for a work note and saves duration into a persistent **Worklogs** table linked to the task.
- A chronological history of recorded work sessions should be displayed on the task.

03. Technical Architecture & Freedom

You have architectural freedom regarding UI styling and schema specifics, but we expect standard Laravel conventions:

- **Relational Design:** Clear migrations and foreign keys connecting Tasks, Checklists, and Worklogs.
- **Eloquent Models:** Well-defined relationships (`hasMany`, `belongsTo`) and proper model encapsulation.
- **Front-End Interaction:** Clean JavaScript (Vanilla JS, Alpine.js, Vue, or Fetch API) with reliable state handling (e.g. `localStorage`).
- **Validation & Security:** Proper request validation, error states, and CSRF protection.

04. Evaluation Rubric

Criteria	Evaluation Focus
Architecture & Code	Adherence to Laravel conventions, clean controllers, readable code, and proper separation of concerns.
Problem Solving	How cleanly you handle stopwatch persistence through page refreshes and dynamic progress recalculations.
UI / UX Quality	Intuitive layout, responsive design, smooth feedback states, and overall attention to usability.
Git Hygiene	Meaningful, incremental commit history demonstrating your iterative development thought process.

05. Submission Instructions

- Push the solution to a public or shared **GitHub / GitLab** repository.
- Include a concise **README.md** outlining setup steps (migrations, seeds, dependencies).
- Send the repository link to our hiring team upon completion.